

VULNERABILITY RESEARCH

C PROGRAMMING

DIFFERENCES

INPUT

```
practice.cpp > main()
1  #include <stdio.h>
2
3  int main() {
4      int i = 3;
5
6      printf("i++: %d\n", i++);
7      printf("After i++, i: %d\n", i);
8
9      i = 4;
10
11     printf("++i: %d\n", ++i);
12     printf("After ++i, i: %d\n", i);
13
14     return 0;
15 }
16
```

In C Language we use both ++ and -- operators. These are called **increment and decrement operators**. They are unary operators only needing one operand.

++i is a pre-increment

i++ is a post-increment

++i will increment i before using it in an expression.

i++ will use the current value of i in the expression first, then increment it.

OUTPUT

```
i++: 3
After i++, i: 4
++i: 5
After ++i, i: 5
```

COMPARE & CONTRAST

Pass-by-value and pass-by-reference are concepts that are important in how arguments are passed to functions. These methods will differ in how they handle function arguments and the changes made within the functions will affect the original variables outside the function.

Pass-by-value: A copy of the actual argument's value is passed to the function, this means that any changes made to the argument within the function will not affect the original value of the argument outside the function. This is usually the default in C.

- The function `modifyValue` takes an integer `a` as an argument >
- When `num` is passed to `modifyValue`, the value 10 is copied to `a` >
- Inside the function, modifying `a` does not affect the original `num` because only the copy is modified >
- Therefore, after the function call, `num` retains its original value

```
practice.cpp > main()
1  #include <stdio.h>
2
3  void modifyValue(int a) {
4      a = 20; // changes the local copy of 'a'
5  }
6
7  int main() {
8      int num = 10;
9      printf("Before: %d\n", num); // output: 10
10
11     modifyValue(num); // passes a copy of 'num'
12
13     printf("After: %d\n", num); // output: 10 (unchanged)
14
15     return 0;
16 }
17
```

COMPARE & CONTRAST

Pass-by-reference: Here, instead of passing a copy of the variable's value, the reference (memory address) of the variable is passed to the function. This allows the function to modify the actual variable, as it works directly with the original data in memory.

- The function `modifyValue` takes a pointer `a`
- The address of `num` is passed to the function using the `&` operator, meaning the function receives the reference to `num`.
- Inside the function, `*a = 20` modifies the value of `num` directly using the pointer.
- After the function call, `num` is modified to 20 because the function had access to its memory location.

```
practice.cpp > main()
1  #include <stdio.h>
2
3  void modifyValue(int *a) {
4      *a = 20; // dereferences pointer 'a' to modify the actual value of 'num'
5  }
6
7  int main() {
8      int num = 10;
9      printf("Before: %d\n", num); // output: 10
10
11     modifyValue(&num); // passes the address of 'num'
12
13     printf("After: %d\n", num); // output: 20 (changed)
14
15     return 0;
16 }
```

KEY DIFFERENCES

ASPECT	PASS-BY-VALUE	PASS-BY-REFERENCE
PARAMETER	A copy of the variable is passed	A reference (pointer) to the original variable is passed
MODIFIES ORIGINAL?	No, the function changes but not the original variable	Yes, changes inside the function will amend the original
MEMORY EFFICENCY	Less efficient for large data structures due a copy being made	More efficient due to only the address being passed and no copy being made
EXAMPLE USAGE	Simplistic data types such as int and float	Used when the function needs to amend original data, such as an array or larger structures
DEFAULT BEHAVIOUR IN C	Pass-by-value for all arguments	Simulated via pointers

DIFFERENCES

== OPERATOR VS STRCMP() FUNCTION

== Operators:

- Used to compare primitive data types like int and char.
- When used with strings == will compare the memory addresses of the strings, not their content.
- This will mean it will check if both pointers point to the same memory location.

```
1  #include <stdio.h>
2
3  char str1[] = "Hello Alison";
4  char str2[] = "Hello Alison";
5  if (str1 == str2){
6      printf("Strings are equal/n");
7  } else {
8      printf("Strings are not equal\n");
9  }
```

The output here will be "Strings are not equal", even though they are (same amount of characters) it's because str1 and str2 point to different memory locations.

strcmp() Function:

- Is specifically designed to compare the content of two strings
- It will use dictionary order to compare the strings and will return:
 - If the strings are equal
 - A negative value if str1 is < str2
 - A positive value if str1 is > than str2

```
1  #include <stdio.h>
2  #include <string.h>
3
4  int main() {
5      char str1[] = "Hello Anisa";
6      char str2[] = "Hello Anisa";
7      if (strcmp(str1, str2) == 0) {
8          printf("Strings are equal\n");
9      } else {
10         printf("Strings are not equal\n");
11     }
12     return 0;
13 }
```

The output here will be "Strings are equal", because strcmp() compares the actual content of both str1 and str2

DIFFERENCES

= OPERATOR AND == OPERATOR

= Operators:

```
15 int a = 5;
```

- Used for **assignment**
- It will assign the value on its right to the variable on its left

== Operators:

```
15 int a = 5;  
16 int b = 5;  
17 if (a == b) {  
18     printf("a and b are equal\n");  
19 }
```

- Used for **comparison**
- It will check if the values of two operands are equal

BUFFER OVERFLOW

HOW STRNCP FUNCTION PREVENTS BUFFER OVERFLOW

By using the `strncpy` function you can prevent buffer overflow vulnerabilities because you can specify the maximum number of characters to copy from a source string to a destination buffer. This will limit the number of bytes that get written to the destination which in turn prevents accidental overwriting of adjacent memory areas unlike when the `strcpy` function is used.

EXAMPLE

```
char dest[10];  
strncpy(dest, "This is a long string", sizeof(dest) - 1); // Safe: only 9 characters copied  
dest[9] = '\0'; // Ensure the string is null-terminated
```


INPUT VALIDATION

Input validation is an important step in ensuring security of applications and for complying with the CIA triad. It is a crucial part of the security practice to make sure that all external data, such as user input, file inputs, data from external systems, are checked and validated before being processed by the programme. This is important for the following reasons:

PREVENT MALICIOUS INPUTS

- Attackers could use SQL Injection, script injection and various other methods to exploit the systems. Correct validation will sanitise the inputs and support avoiding security breaches.

PROTECT AGAINST COMMON ATTACKS

- By validating the input it would protect the data against common attacks such as XSS,SQL/Script injections, buffer overflows, command injection etc

MAINTAIN DATA INTEGRITY

- Ensuring that input is will make sure that the system behaves as it should and would prevent invalid data from corrupting application logic or databases.

AVOIDS UNINTENDED BEHAVIOUR

- Without proper validation, incorrect input could not only compromise security but also could lead to crashes, undefined behaviour, system failures and instability and errors in data processing.

INPUT VALIDATION

One of the vulnerabilities with input validation would be a SQL Injection. An attacker could manipulate inputs by injecting malicious code. Input validation can prevent this by only allowing valid, expected inputs.

EXAMPLE WITHOUT VALIDATION

```
11 char query [256];
12 sprintf(query, "SELECT * FROM users WHERE username='%s' AND password= '%s' ;", username, password);
13 execute_query(query);
```

An attacker can add an input (“**admin' --**”) that would result in the SQL query to override the AND clause which would bypass authentication, login as an admin and to gain access without needing a valid password.

EXAMPLE WITH VALIDATION

```
15 const char *query = "SELECT * FROM users WHERE username=? AND password=?";
16 prepared_stmt = prepare_statement(connection, query);
17 bind_parameters(prepared_stmt, username, password);
18 execute_query(prepared_stmt);
```

The username and password inputs are now correctly validated. The SQL syntax can not be manipulated or malicious input added as the inputs are treated as data instead of a query.

INPUT

```

1  #include <stdio.h>
2
3  int main() {
4      double num1, num2, result;
5      char operator;
6
7      // Get the user to enter two numbers
8      printf("Enter the first number: ");
9      scanf("%lf", &num1);
10     printf("Enter the second number: ");
11     scanf("%lf", &num2);
12
13     // User choses an operation
14     printf("Choose an operation (+, -, *, /): ");
15     scanf(" %c", &operator);
16
17     // Execute the operation and handle invalid inputs
18     switch (operator) {
19         case '+':
20             result = num1 + num2;
21             printf("Result: %.2lf + %.2lf = %.2lf\n", num1, num2, result);
22             break;
23         case '-':
24             result = num1 - num2;
25             printf("Result: %.2lf - %.2lf = %.2lf\n", num1, num2, result);
26             break;
27         case '*':
28             result = num1 * num2;
29             printf("Result: %.2lf * %.2lf = %.2lf\n", num1, num2, result);
30             break;
31         case '/':
32             if (num2 !=0) {
33                 result = num1/num2;
34                 printf("Result: %.2lf / %.2lf = %.2lf\n", num1, num2, result);
35             } else {
36                 printf("Error: Division by zero is not allowed,\n");
37             }
38             break;
39         default:
40             printf("Error: Invalid operation. Please use +, -, *, or /\n");
41     }
42
43     return 0;
44 }

```

CALCULATOR WRITTEN IN C

Output

```
/tmp/Sk7PSH6M4g.o
```

```
Enter the first number: 3
```

```
Enter the second number: 100
```

```
Choose an operation (+, -, *, /): /
```

```
Result: 3.00 / 100.00 = 0.03
```

```
=== Code Execution Successful ===
```